

CSCE 670 INFORMATION RETRIEVAL · MODULE 06 · LECTURE NOTES

Learning to Rank

Ranking as a supervised problem.

MODULE OUTCOMES

- Explain why ranking is not classification or regression.
- Explain the three loss formulations.
- Explain the features that matter and where they come from.
- Explain the two-stage architecture and why it exists.
- Train and deploy a ranker.

1 Why ranking is its own problem

RANKING IS NOT CLASSIFICATION, BECAUSE THE LOSS DEPENDS ON THE OTHER DOCUMENTS

Classifying each document independently as relevant or not ignores the fact that **only the order matters** — and **getting the top result right matters very much more than getting the hundredth right**, which a per-document loss cannot express.

So the loss does not decompose over documents: a document's contribution to the quality of the result depends on where all the other documents ended up, which ordinary supervised learning assumes away entirely.

And the metrics are not differentiable. NDCG depends on the sorted order of the scores, so it is piecewise constant as a function of the scores and **has zero gradient almost everywhere** — which means it cannot be optimised directly by gradient descent.

Which is exactly why there are three formulations (§1's table) — **each one a different way of getting a usable gradient out of a ranking objective** — and **the differences between them are about that rather than about features**, which is worth keeping straight.

Approach	What it learns	The trade
Pointwise	A relevance score for each document independently.	Simplest, and it ignores rank entirely — so it spends equal effort on position 1 and position 1000.
Pairwise	Which of two documents should rank higher.	Natural to derive from clicks (Module 05 §4), and it weights all pairs equally, which is §2's problem.
Listwise	Optimises a metric computed over the whole result list.	Matches the actual objective, and is harder to optimise because of the differentiability problem above.

LambdaMART is the practical answer, and it sits oddly across this taxonomy: **a pairwise gradient weighted by how much swapping the pair would change NDCG, which is listwise in effect and pairwise in form. Its trick is the thing to understand in this module**, because it is both the practical default and a genuinely clever resolution of a real difficulty.

2 LambdaMART

WEIGHT EACH PAIRWISE GRADIENT BY THE NDCG CHANGE THAT SWAPPING THE PAIR WOULD CAUSE

A plain pairwise model treats every misordered pair as equally important — but swapping positions 1 and 2 matters enormously to the user and swapping positions 99 and 100 matters essentially not at all, and the metric knows this while the loss does not.

So multiply each pair's gradient by the absolute change in NDCG that swapping them would produce — which makes the model concentrate its learning effort precisely where the metric is sensitive, and costs almost nothing to compute.

Which yields a gradient that effectively optimises a non-differentiable metric — by construction rather than by constructing a differentiable approximation to the metric, which is what the listwise methods attempt and which works less well.

Combined with gradient-boosted decision trees, this is LambdaMART — and it remains extremely strong on tabular ranking features (CSCE 633 Module 07's finding that boosted trees dominate tabular problems, arriving here), **which is why it is still deployed very widely** in systems that also have neural components.

3 Features

QUERY-DOCUMENT MATCH

BM25 per field (title, body, anchor text), term proximity, coverage of the query terms, and any semantic similarity score (Module 07)

DOCUMENT QUALITY, independent of the query

PageRank or another link-based score (Module 10), length, spam score, freshness, readability, domain reputation

QUERY FEATURES

length, rarity, detected intent (Module 09)

BEHAVIOURAL

historical click-through rate for this query-document pair, dwell time, skip rate, and aggregate document popularity

AND THE BEHAVIOURAL FEATURES ARE THE STRONGEST and also the most dangerous: they encode what was previously shown, which creates the feedback loop of Module 12 section 3, and they are entirely unavailable for anything new.

- **Feature engineering beat model sophistication** for most of this field's history — and **a good feature set with a simple model beats a sophisticated model with a poor feature set**, reliably, which is worth knowing before choosing where to spend effort.
- **Features must be computable within the latency budget, for every candidate document** — which **excludes a great deal** that would otherwise be useful (Module 04 §4), and is why the two-stage architecture of §4 matters so much.
- **And they must be available at serving time**, which excludes anything derived from the future or from the complete session — **which is CSCE 633 Module 11's leakage problem**, and it is easy to violate accidentally when training from logs.
- **Watch for position bias leaking in through the behavioural features** (Module 05 §4), **which encodes the old ranking directly into the new one** and is the mechanism behind Module 12 §3's feedback loop.
- **And log the features as served, rather than recomputing them for training** — because **recomputation drifts as the pipeline changes and the training data silently becomes wrong**. **Logging features as served is the practice that prevents a slow, invisible training/serving skew**, which is among the hardest production problems to diagnose after the fact.

4 The two-stage architecture

RETRIEVE CHEAPLY, THEN RERANK EXPENSIVELY — BECAUSE THE BUDGET IS PER QUERY AND NOT PER DOCUMENT

A good ranker is far too expensive to run over a million documents — but running the same ranker over a hundred candidates is entirely affordable, and a hundred is enough *if the right documents are among them*.

So: a cheap, high-recall first stage — BM25, or dense retrieval, or both combined (Module 07 §4) — followed by an expensive, high-precision reranker over its top few hundred results. This is the shape of essentially every production search system.

Which makes first-stage recall the binding constraint: the reranker cannot recover a document the retrieval stage missed — which is exactly CSCE 638 Module 11 §2's point, and is why §4's first deployment measurement is first-stage recall.

And the number of candidates is the dial — more candidates means better recall and a more expensive reranking step — which is precisely where the latency budget enters the design rather than being discovered afterwards (Module 04 §4).

- **Measure recall at the first-stage cutoff**, separately from the end-to-end quality — because **it bounds everything downstream** and a system failing here cannot be fixed by improving the reranker.
- **Measure the reranker's effect on the candidates it actually received**, which **isolates its contribution** from the retrieval's and tells you which half to invest in.
- **Retrain on a schedule**, because **the collection, the query distribution, and the behavioural features all drift** (CSCE 633 Module 11) — and the behavioural features drift fastest, since they depend on what you have been showing.
- **And monitor the feature distributions**, since **a silently broken feature degrades quality without producing any error at all** — the model simply gets a constant or a default where it expected a signal, and keeps serving.
- **Which is the commonest production failure in a learned ranker**: a feature pipeline breaks upstream and the model keeps serving, somewhat worse, indefinitely. **A silently broken feature is the characteristic learned-ranker failure** — it degrades gracefully and nothing alerts — **which is why feature monitoring is the control that matters** rather than model monitoring.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Liu — Learning to Rank for Information Retrieval (free survey)	§1's three formulations , organised, with the theoretical relationships between them.

Resource	What it covers for this module
Burges — From RankNet to LambdaRank to LambdaMART (free)	§2 by its author — the clearest account of the lambda trick and why it works.
Qin et al. — Are Neural Rankers still Outperformed by Gradient Boosted Decision Trees? (free)	§2's continued relevance, measured — on tabular ranking features, which is the setting that matters here.
Joachims et al. — Unbiased Learning-to-Rank with Biased Feedback (free)	§3's position bias problem, with the correction — inverse propensity weighting for click data.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Explain why a per-document loss is wrong** for ranking, with a concrete example.
- 2 **Show that NDCG has zero gradient** almost everywhere.
- 3 **Implement pointwise, pairwise, and listwise losses** and compare their NDCG.
- 4 **Add the lambda weighting to your pairwise loss** and measure the improvement.
- 5 **Build a feature set** with at least one feature from each of the four groups.
- 6 **Train LambdaMART** and measure feature importance.
- 7 **Remove the behavioural features** and report what you lose.
- 8 **Build the two-stage architecture** and measure first-stage recall at 100 and 1000.
- 9 **Measure the reranker's effect** on the candidates it received.
- 10 **Break one feature deliberately** and measure how much quality falls before anything alerts.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why is ranking not classification?
- 2 Why are the metrics not differentiable?
- 3 Name the three formulations and the trade in each.
- 4 Explain LambdaMART's weighting and what problem it solves.
- 5 Name the four feature groups with an example from each.
- 6 Why are behavioural features strongest and most dangerous?

- 7 Give five practical findings about features.
- 8 Why log features as served?
- 9 State the two-stage argument and what the binding constraint is.
- 10 Name the characteristic production failure and the control for it.